

Weather Impact on Public Transport Usage

This analysis explores how weather conditions, particularly rainfall, affect public transport activity.

The goal is to identify patterns in transport usage during rainy and non-rainy conditions using historical transport and climate data.

Author: Rukshan Dias

Student ID: s224326349

Level: Undergraduate ? Junior Duration: Weeks 1–4

Pre-requisite Skills: Python, Pandas, Data Analysis

```
In [1]: import pandas as pd
import os

RAW_PATH = r"C:\Users\User\Desktop\SIT374\traffic_project\data\raw"
os.chdir(RAW_PATH)

print("Current directory:", os.getcwd())
print("Files:", os.listdir())
```

```
Current directory: C:\Users\User\Desktop\SIT374\traffic_project\data\raw
Files: ['bom_weather_2025.csv', 'TransportActivityCount_2025_1.csv', 'TransportActiv
ityCount_2025_2.csv', 'TransportActivityCount_2025_3.csv', 'TransportActivityCount_2
025_4.csv', 'venues-for-event-bookings.csv']
```

Load Datasets

We load the transport activity dataset and climate dataset for analysis.

```
In [3]: t1 = pd.read_csv("TransportActivityCount_2025_1.csv")
t2 = pd.read_csv("TransportActivityCount_2025_2.csv")
t3 = pd.read_csv("TransportActivityCount_2025_3.csv")
t4 = pd.read_csv("TransportActivityCount_2025_4.csv")

traffic = pd.concat([t1, t2, t3, t4], ignore_index=True)

print("Traffic shape:", traffic.shape)
traffic.head()
```

```
Traffic shape: (5421538, 8)
```

Out[3]:

	countLocationId	countLocationName	CountLocationLat	CountLocationLong	
0	51228	Yarra Promenade - East Path	-37.821459	144.960343	20201T00:00:00
1	45217	Royal Park Tram Track- CoM Pole 2773	-37.791020	144.949562	20201T00:00:00
2	43775	Swanston St Asset ID: COM1757	-37.808159	144.963052	20201T00:00:00
3	44864	Royal Park Asset ID:COM2705	-37.795462	144.952063	20201T00:00:00
4	51281	Yarra Promenade - West Path	-37.822354	144.958747	20201T00:00:00

```
In [4]: weather = pd.read_csv("bom_weather 2025.csv")

print("Weather shape:", weather.shape)
weather.head()
```

Weather shape: (365, 8)

Out[4]:

	Product code	Bureau of Meteorology station number	Year	Month	Day	Rainfall amount (millimetres)	Period over which rainfall was measured (days)	Quality
0	IDCJAC0009	86338	2025	1	1	0.0	1.0	Y
1	IDCJAC0009	86338	2025	1	2	0.0	1.0	Y
2	IDCJAC0009	86338	2025	1	3	0.0	1.0	Y
3	IDCJAC0009	86338	2025	1	4	0.0	1.0	Y
4	IDCJAC0009	86338	2025	1	5	0.0	1.0	Y

Data Cleaning

Remove duplicates and handle missing values.

```
In [5]: traffic.drop_duplicates(inplace=True)
weather.drop_duplicates(inplace=True)

traffic = traffic.ffill()
weather = weather.ffill()

print("Traffic missing values:")
print(traffic.isnull().sum())
```

```
print("\nWeather missing values:")
print(weather.isnull().sum())
```

Traffic missing values:

```
countLocationId      0
countLocationName     0
CountLocationLat      0
CountLocationLong     0
from                  0
to                    0
class                 0
count                 0
dtype: int64
```

Weather missing values:

```
Product code          0
Bureau of Meteorology station number  0
Year                  0
Month                 0
Day                   0
Rainfall amount (millimetres)  0
Period over which rainfall was measured (days)  0
Quality                0
dtype: int64
```

Feature Engineering

Prepare time and weather features for analysis.

Code (FIX DATE + TIME — IMPORTANT

```
In [6]: # Convert traffic time
traffic['from'] = pd.to_datetime(traffic['from'])

# Extract hour
traffic['hour'] = traffic['from'].dt.hour

# Create date column
traffic['date'] = traffic['from'].dt.date
traffic['date'] = pd.to_datetime(traffic['date'])
```

Code (FIX WEATHER DATE — IMPORTANT)

```
In [7]: weather['date'] = pd.to_datetime(weather[['Year', 'Month', 'Day']])

weather = weather.rename(columns={
    'Rainfall amount (millimetres)': 'rainfall'
})
```

Merge Data

Combine traffic and weather datasets using the date column.

```
In [8]: data = pd.merge(
    traffic,
    weather[['date', 'rainfall']],
    on='date',
    how='left'
)

print("Merged shape:", data.shape)
print(data.head())
print("\nMissing values after merge:")
print(data.isnull().sum())
```

```
Merged shape: (4721358, 11)
   countLocationId      countLocationName  CountLocationLat \
0           51228      Yarra Promenade - East Path      -37.821459
1          45217  Royal Park Tram Track- CoM Pole 2773      -37.791020
2          43775      Swanston St Asset ID: COM1757      -37.808159
3          44864      Royal Park Asset ID:COM2705      -37.795462
4           51281      Yarra Promenade - West Path      -37.822354

   CountLocationLong      from      to \
0      144.960343  2025-01-01 00:00:00+00:00  2025-01-01T00:05:00.000Z
1      144.949562  2025-01-01 00:00:00+00:00  2025-01-01T00:05:00.000Z
2      144.963052  2025-01-01 00:00:00+00:00  2025-01-01T00:05:00.000Z
3      144.952063  2025-01-01 00:00:00+00:00  2025-01-01T00:05:00.000Z
4      144.958747  2025-01-01 00:00:00+00:00  2025-01-01T00:05:00.000Z

   class  count  hour      date  rainfall
0  pedestrian    313    0  2025-01-01      0.0
1  pedestrian     1    0  2025-01-01      0.0
2    cyclist     1    0  2025-01-01      0.0
3  pedestrian     2    0  2025-01-01      0.0
4  pedestrian    61    0  2025-01-01      0.0

Missing values after merge:
countLocationId      0
countLocationName    0
CountLocationLat     0
CountLocationLong    0
from                 0
to                   0
class                0
count                0
hour                 0
date                 0
rainfall             0
dtype: int64
```

Exploratory Data Analysis

In this section, we explore the relationship between rainfall and public transport usage.

The aim is to identify patterns, trends, and possible correlations before building a prediction model.

```
In [9]: print("Columns in merged dataset:")
        print(data.columns)

        print("\nBasic info:")
        print(data.info())

        print("\nSummary statistics:")
        print(data.describe(include='all'))
```

```
Columns in merged dataset:
Index(['countLocationId', 'countLocationName', 'CountLocationLat',
      'CountLocationLong', 'from', 'to', 'class', 'count', 'hour', 'date',
      'rainfall'],
      dtype='str')
```

```
Basic info:
<class 'pandas.DataFrame'>
RangeIndex: 4721358 entries, 0 to 4721357
Data columns (total 11 columns):
#   Column                Dtype
---  -
0   countLocationId      int64
1   countLocationName    str
2   CountLocationLat     float64
3   CountLocationLong    float64
4   from                 datetime64[us, UTC]
5   to                   str
6   class                str
7   count                int64
8   hour                 int32
9   date                 datetime64[s]
10  rainfall              float64
dtypes: datetime64[s](1), datetime64[us, UTC](1), float64(3), int32(1), int64(2), str(3)
memory usage: 378.2 MB
None
```

Summary statistics:

	countLocationId	countLocationName	CountLocationLat	\
count	4.721358e+06	4721358	4.721358e+06	
unique	NaN	23	NaN	
top	NaN	La Trobe St- William St I-Hub	NaN	
freq	NaN	1376018	NaN	
mean	4.638385e+04	NaN	-3.780908e+01	
min	4.377200e+04	NaN	-3.782392e+01	
25%	4.504000e+04	NaN	-3.782018e+01	
50%	4.547200e+04	NaN	-3.781192e+01	
75%	4.641700e+04	NaN	-3.780816e+01	
max	5.809700e+04	NaN	-3.777977e+01	
std	2.677084e+03	NaN	1.338081e-02	

	CountLocationLong	from	\
count	4.721358e+06	4721358	
unique	NaN	NaN	
top	NaN	NaN	
freq	NaN	NaN	
mean	1.449598e+02	2025-07-05 12:28:15.970356+00:00	
min	1.449415e+02	2025-01-01 00:00:00+00:00	
25%	1.449563e+02	2025-04-05 02:10:00+00:00	
50%	1.449603e+02	2025-07-06 19:15:00+00:00	
75%	1.449626e+02	2025-10-05 23:05:00+00:00	
max	1.449876e+02	2025-12-31 23:55:00+00:00	
std	6.495471e-03	NaN	

	to	class	count	hour	\
--	----	-------	-------	------	---

count	4721358	4721358	4.721358e+06	4.721358e+06
unique	105018	14	NaN	NaN
top	2025-12-11T09:00:00.000Z	pedestrian	NaN	NaN
freq	86	1861620	NaN	NaN
mean	NaN	NaN	1.345293e+01	1.271867e+01
min	NaN	NaN	1.000000e+00	0.000000e+00
25%	NaN	NaN	1.000000e+00	8.000000e+00
50%	NaN	NaN	3.000000e+00	1.300000e+01
75%	NaN	NaN	1.100000e+01	1.800000e+01
max	NaN	NaN	1.904000e+03	2.300000e+01
std	NaN	NaN	3.280250e+01	6.033426e+00

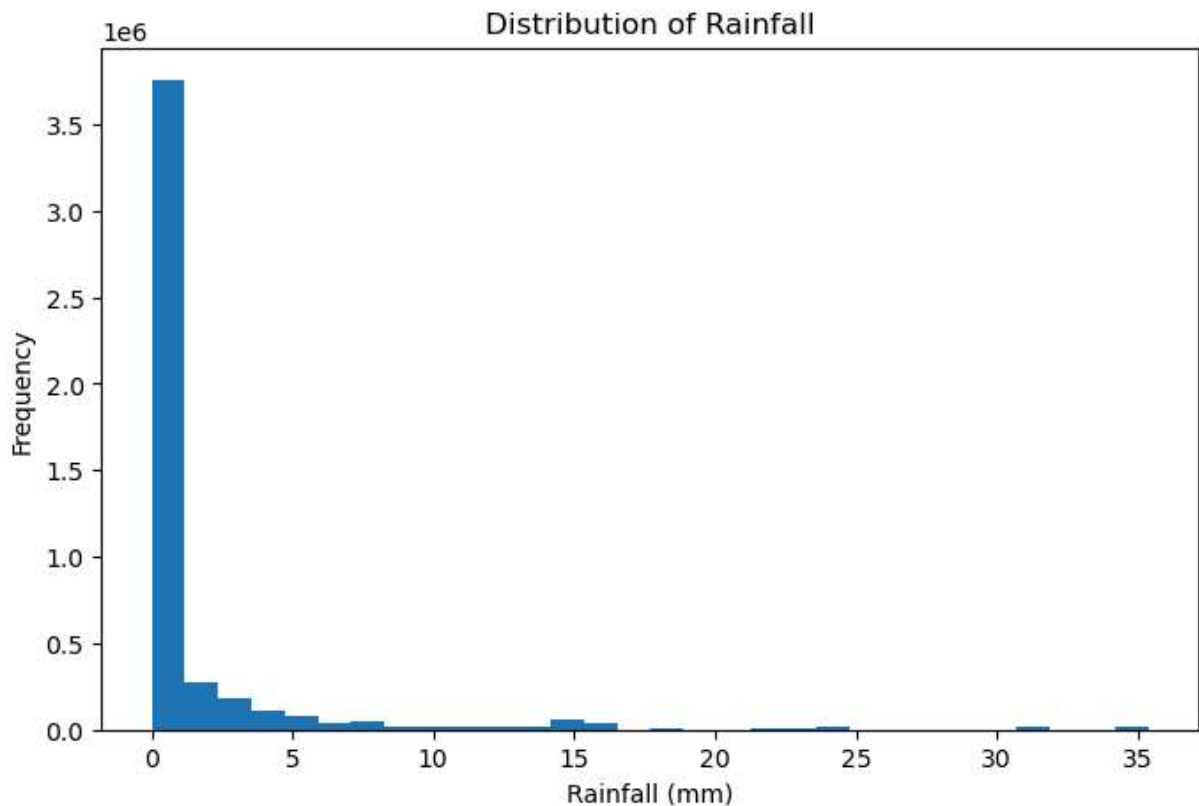
	date	rainfall
count	4721358	4.721358e+06
unique	NaN	NaN
top	NaN	NaN
freq	NaN	NaN
mean	2025-07-04 23:17:35	1.453014e+00
min	2025-01-01 00:00:00	0.000000e+00
25%	2025-04-05 00:00:00	0.000000e+00
50%	2025-07-06 00:00:00	0.000000e+00
75%	2025-10-05 00:00:00	6.000000e-01
max	2025-12-31 00:00:00	3.540000e+01
std	NaN	4.187549e+00

Rainfall Distribution

This plot shows how rainfall values are distributed across the dataset. Understanding the spread of rainfall helps identify whether the data is mostly dry days or includes a reasonable range of wet conditions.

```
In [10]: import matplotlib.pyplot as plt

plt.figure(figsize=(8, 5))
plt.hist(data['rainfall'].dropna(), bins=30)
plt.xlabel('Rainfall (mm)')
plt.ylabel('Frequency')
plt.title('Distribution of Rainfall')
plt.show()
```



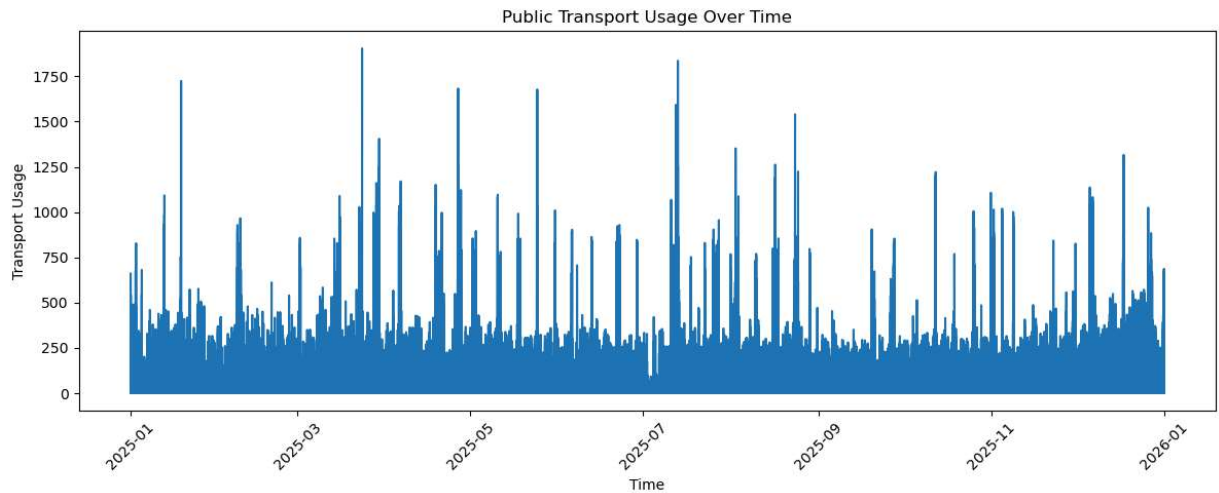
Public Transport Usage Over Time

This visualization shows how transport usage changes over time. It helps identify patterns such as peaks, drops, and general variability in transport activity.

```
In [14]: print(data.columns)
```

```
Index(['countLocationId', 'countLocationName', 'CountLocationLat',  
      'CountLocationLong', 'from', 'to', 'class', 'count', 'hour', 'date',  
      'rainfall'],  
      dtype='str')
```

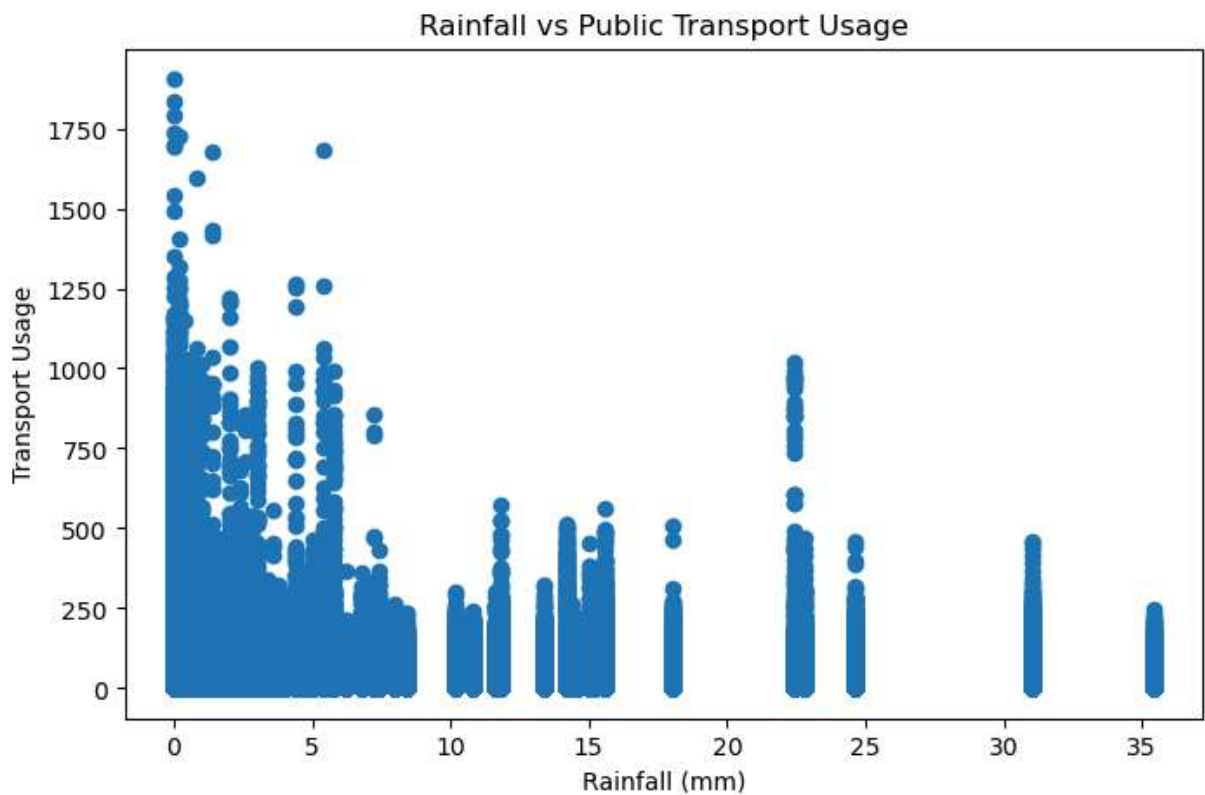
```
In [12]: plt.figure(figsize=(12, 5))  
plt.plot(data['from'], data['count'])  
plt.xlabel('Time')  
plt.ylabel('Transport Usage')  
plt.title('Public Transport Usage Over Time')  
plt.xticks(rotation=45)  
plt.tight_layout()  
plt.show()
```



Rainfall vs Public Transport Usage

This scatter plot helps examine whether rainfall appears to influence public transport usage. If a visible trend exists, it suggests that weather may have an effect on transport demand.

```
In [13]: plt.figure(figsize=(8, 5))
plt.scatter(data['rainfall'], data['count'])
plt.xlabel('Rainfall (mm)')
plt.ylabel('Transport Usage')
plt.title('Rainfall vs Public Transport Usage')
plt.show()
```



This scatter plot shows a weak relationship between rainfall and public transport usage.

The data points are widely scattered, indicating that rainfall alone does not strongly determine transport activity. While some lower transport usage is observed at higher rainfall levels, the overall pattern is not clearly linear.

This suggests that additional factors, such as time of day or location, may have a stronger influence on transport usage.

Average Transport Usage on Rainy and Non-Rainy Days

To simplify interpretation, rainfall is converted into two categories:

- Non-rainy day
- Rainy day

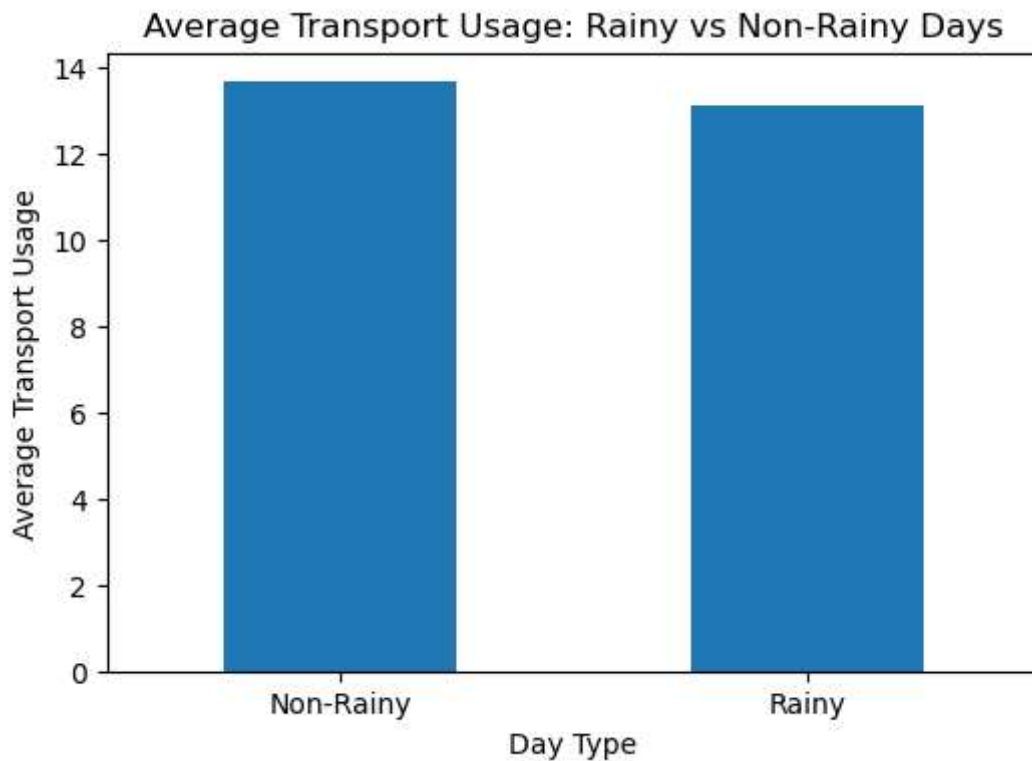
This helps compare the average transport usage under each condition.

```
In [15]: data['rainy_day'] = data['rainfall'].apply(lambda x: 1 if x > 0 else 0)

avg_usage = data.groupby('rainy_day')['count'].mean()
print(avg_usage)

plt.figure(figsize=(6, 4))
avg_usage.plot(kind='bar')
plt.xticks([0, 1], ['Non-Rainy', 'Rainy'], rotation=0)
plt.xlabel('Day Type')
plt.ylabel('Average Transport Usage')
plt.title('Average Transport Usage: Rainy vs Non-Rainy Days')
plt.show()
```

```
rainy_day
0    13.646358
1    13.102623
Name: count, dtype: float64
```



The comparison shows that average transport usage is very similar for both rainy and non-rainy conditions.

This indicates that rainfall does not significantly impact overall transport usage in this dataset. The difference between the two categories is minimal, suggesting that other factors such as time or daily patterns may play a more important role.

Model Building

A simple machine learning model is developed to predict public transport usage based on weather and time-related features.

For this analysis, the following features are used:

- rainfall
- hour

The target variable is:

- public transport usage

```
In [16]: TARGET = 'count'
```

```
In [17]: model_data = data[['rainfall', 'hour', TARGET]].copy()
model_data = model_data.dropna()
```

```
print("Model dataset shape:", model_data.shape)
model_data.head()
```

Model dataset shape: (4721358, 3)

```
Out[17]:
```

	rainfall	hour	count
0	0.0	0	313
1	0.0	0	1
2	0.0	0	1
3	0.0	0	2
4	0.0	0	61

Train-Test Split

The dataset is divided into training and testing sets. The training set is used to train the model, while the testing set is used to evaluate its predictive performance.

```
In [18]: from sklearn.model_selection import train_test_split

X = model_data[['rainfall', 'hour']]
y = model_data[TARGET]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print("Training set size:", X_train.shape)
print("Testing set size:", X_test.shape)
```

Training set size: (3777086, 2)

Testing set size: (944272, 2)

Linear Regression Model

A Linear Regression model is used as a baseline model to predict transport usage.

This model is suitable for understanding whether there is a simple linear relationship between rainfall, hour, and transport activity.

```
In [19]: from sklearn.linear_model import LinearRegression

model = LinearRegression()
model.fit(X_train, y_train)

print("Model training completed.")
```

Model training completed.

Model Evaluation

The model is evaluated using:

- Mean Absolute Error (MAE)
- Mean Squared Error (MSE)
- R-squared (R^2)

These metrics help measure how accurately the model predicts transport usage.

```
In [20]: from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

y_pred = model.predict(X_test)

mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print("MAE:", mae)
print("MSE:", mse)
print("R²:", r2)
```

```
MAE: 15.848650408208922
MSE: 1077.7763058328387
R²: 0.009200333700574492
```

Model Performance Interpretation

The model shows a very low R-squared value, indicating that it explains only a small portion of the variance in transport usage.

This suggests that rainfall and hour alone are not sufficient predictors of transport activity. While the model provides a basic baseline, it does not capture the complexity of real-world transport patterns.

Additional features such as location, day of the week, or special events would likely improve model performance.

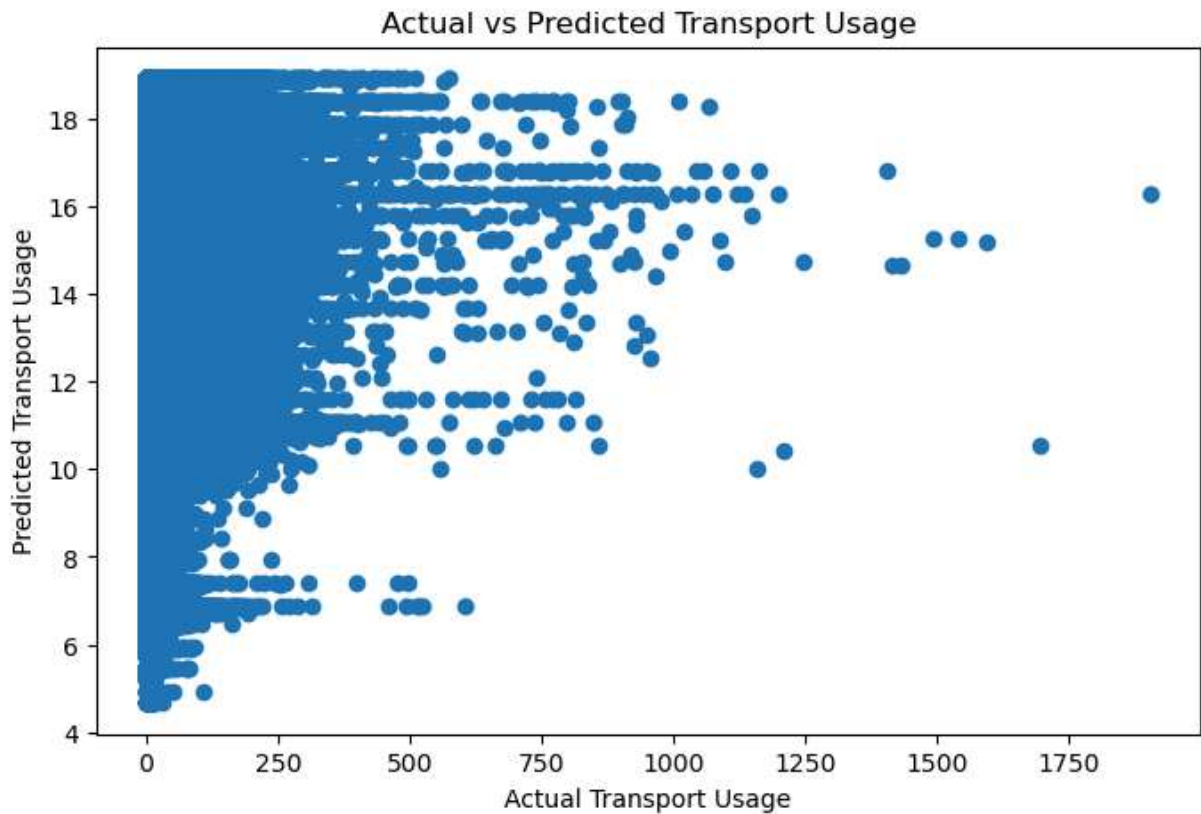
Actual vs Predicted Values

This plot compares the actual transport usage values with the predicted values from the model.

A closer alignment between the two indicates better model performance.

```
In [21]: plt.figure(figsize=(8, 5))
plt.scatter(y_test, y_pred)
plt.xlabel("Actual Transport Usage")
```

```
plt.ylabel("Predicted Transport Usage")
plt.title("Actual vs Predicted Transport Usage")
plt.show()
```



The scatter plot shows a weak alignment between actual and predicted values.

The predictions are widely dispersed and do not closely follow the actual values, indicating that the model has limited predictive accuracy.

This further confirms that the current feature set is insufficient to accurately model transport usage.

Model Coefficients

The coefficients of the Linear Regression model show the direction and strength of the relationship between each feature and transport usage.

```
In [22]: coefficients = pd.DataFrame({
    'Feature': X.columns,
    'Coefficient': model.coef_
})

print(coefficients)
```

	Feature	Coefficient
0	rainfall	-0.062080
1	hour	0.523547

Key Findings

The analysis reveals that rainfall has a limited impact on public transport usage.

Exploratory analysis showed no strong relationship between rainfall and transport activity, and the difference between rainy and non-rainy conditions was minimal.

The machine learning model further confirmed this, with a very low R-squared value indicating weak predictive power.

Time-based features such as hour appear to have a greater influence on transport usage compared to weather conditions alone.

Overall, transport patterns are likely influenced by multiple factors beyond rainfall.

Conclusion

This project analysed the impact of weather conditions on public transport usage using transport activity and climate data.

The datasets were successfully cleaned, merged, and explored through visualisations and statistical analysis. A baseline Linear Regression model was then developed to predict transport usage using rainfall and hour as features.

The results indicate that rainfall has only a minor influence on transport usage, while time-related factors appear to play a more significant role. The model's low predictive performance highlights the limitations of using a small feature set.

Future improvements could include incorporating additional variables such as day of the week, location-based features, and real-world event data with timestamps.

This study demonstrates that urban transport behaviour is complex and requires a combination of environmental and temporal factors for accurate prediction.